SIMATS ENGINEERING

CO3 ASSESSMENT TOOL 1 - EXPERIMENTS

NAME : HITESH C

REG NO : 192524005

COURSE CODE : CSA1401

COURSE NAME : COMPILER DESIGN

COURSE FACULTY : AJITHA V

QUESTION 1

Q1. A calculator compiler evaluates arithmetic expressions during parsing using YACC.
*(i) Design a YACC grammar for arithmetic operators + and *.*
(ii) Embed semantic actions in the grammar rules to compute expression values.
*(iii) Evaluate the input expression: 3 + 4 * 5*
(iv) Display the computed result after parsing. (10 Marks)

ANSWER FOR QUESTION 1

(i) & (ii) Lexer and YACC Grammar with Embedded Semantic Actions

1. Lexer Specification (calc.l):

```
%{
#include "y.tab.h"
#include <stdio.h>
%}

%%
[0-9]+  { yylval = atoi(yytext); return NUMBER; }
[+\*\n] { return yytext[0]; }
[ \t]   { /* ignore whitespace */ }
.       { printf("Invalid character: %s\n", yytext); }
%%
```

```
int yywrap() { return 1; }
```

2. Parser Specification with Semantic Actions (calc.y):

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
%}

%token NUMBER
%left '+'
%left '*'

%%
input:
    expr '\n' { printf("Computed Result = %d\n", $1); exit(0); }
    ;

expr:
    expr '+' expr { $$ = $1 + $3; }    /* Rule 1: Addition */
  | expr '*' expr { $$ = $1 * $3; }    /* Rule 2: Multiplication */
  | NUMBER        { $$ = $1; }         /* Rule 3: Primitive Number */
  ;
%%

void yyerror(const char *s) {
    fprintf(stderr, "Parse Error: %s\n", s);
}

int main() {
    printf("Enter expression: ");
    yyparse();
    return 0;
}
```

(iii) Evaluation of Input Expression: 3 + 4 * 5

Operator Precedence Analysis: The declaration %left '*' below %left '+' specifies that multiplication '*' has a higher precedence than addition '+'. Therefore, '4 * 5' is grouped and evaluated before adding '3'.

Step-by-Step Parse Stack and Reduction Trace:

1. Shift '3' onto the stack.

2. Reduce '3' using Rule 3 (expr -> NUMBER). Attribute value synthesized: $$ = 3.
3. Shift '+' onto stack.
4. Shift '4' onto stack.
5. Reduce '4' using Rule 3 (expr -> NUMBER). Attribute value synthesized: $$ = 4.
6. Lookahead is '*'. Since '*' has higher precedence than '+', the parser SHIFTS '*' instead of reducing '3 + 4'.
7. Shift '5' onto stack.
8. Reduce '5' using Rule 3 (expr -> NUMBER). Attribute value synthesized: $$ = 5.
9. Reduce 'expr(4) * expr(5)' using Rule 2 (expr -> expr * expr). Action: $$ = 4 * 5 = 20.
10. Reduce 'expr(3) + expr(20)' using Rule 1 (expr -> expr + expr). Action: $$ = 3 + 20 = 23.

(iv) Computed Result Displayed

The parser completes successfully and outputs:

Computed Result = 23

—

QUESTION 2

Q2. A compiler constructs syntax trees for subsequent semantic analysis and code generation using YACC.
(i) Define a YACC grammar for arithmetic expressions.
(ii) Implement semantic actions to construct a parse tree / Abstract Syntax Tree (AST).
*(iii) Generate and illustrate the tree for: a + b * c*
(iv) Traverse the generated tree and display the traversal order. (10 Marks)

ANSWER FOR QUESTION 2

(i) & (ii) YACC Grammar & Semantic Actions for Constructing AST

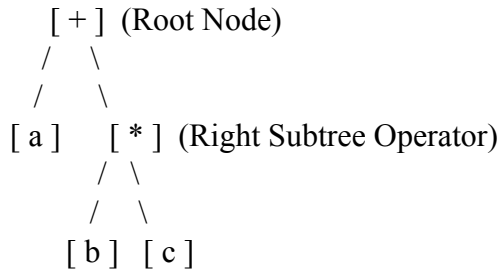
AST Node Structure definition and YACC specification (ast.y):

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct ASTNode {
   char label[10];
   struct ASTNode *left;
```

```
    struct ASTNode *right;
} ASTNode;

ASTNode* createNode(char* label, ASTNode* left, ASTNode* right) {
    ASTNode* node = (ASTNode*)malloc(sizeof(ASTNode));
    strcpy(node->label, label);
    node->left = left;
    node->right = right;
    return node;
}

#define YYSTYPE ASTNode*
int yylex();
void yyerror(const char *s);
void printPreorder(ASTNode* root);
void printInorder(ASTNode* root);
void printPostorder(ASTNode* root);
%}

%token IDENTIFIER
%left '+'
%left '*'

%%
stmt:
    expr {
        printf("\nAST Successfully Built!\n");
        printf("Pre-order Traversal: "); printPreorder($1); printf("\n");
        printf("In-order Traversal:  "); printInorder($1); printf("\n");
        printf("Post-order Traversal:"); printPostorder($1); printf("\n");
    }
    ;

expr:
    expr '+' expr { $$ = createNode("+", $1, $3); }
  | expr '*' expr { $$ = createNode("*", $1, $3); }
  | IDENTIFIER    { $$ = $1; }
  ;
%%
```

(iii) AST Generation and Structural Illustration for: a + b * c

For the expression 'a + b * c', '*' has higher precedence than '+'. The compiler constructs the following tree hierarchy:



Node Details:
- Root Node: Operator '+'
- Left Child of '+': Leaf Node 'a'
- Right Child of '+': Internal Node operator '*'
- Left Child of '*': Leaf Node 'b'
- Right Child of '*': Leaf Node 'c'

(iv) Tree Traversals and Display Orders

The generated AST is traversed using three standard traversal algorithms:

1. Pre-order Traversal (Root -> Left -> Right):
   Algorithm: Visit Root, traverse Left Subtree, traverse Right Subtree.
   Traversal Sequence: + a * b c
   Compiler Use: Prefix expression / Abstract syntax tree representation.

2. In-order Traversal (Left -> Root -> Right):
   Algorithm: Traverse Left Subtree, Visit Root, traverse Right Subtree.
   Traversal Sequence: a + b * c
   Compiler Use: Reconstructing fully parenthesized source expression.

3. Post-order Traversal (Left -> Right -> Root):
   Algorithm: Traverse Left Subtree, traverse Right Subtree, Visit Root.
   Traversal Sequence: a b c * +
   Compiler Use: Bottom-up evaluation and Intermediate Code Generation (Postfix/Reverse Polish Notation).

QUESTION 3

Q3. A compiler evaluates arithmetic expressions using synthesized attributes only in a YACC-based syntax-directed translation scheme.
(i) Implement a Syntax-Directed Definition (SDD) using YACC semantic values ($$, $1, $2, etc.).
*(ii) Evaluate the expression: 2 * 3 + 4*

(iii) Show how attribute values are propagated during reductions.
(iv) Demonstrate the bottom-up evaluation process performed by the parser. (10 Marks)

ANSWER FOR QUESTION 3

(i) Syntax-Directed Definition (SDD) with Synthesized Attributes

An S-attributed SDD relies exclusively on synthesized attributes. In YACC, semantic values $$ represent the attribute of the head (LHS), and $1, $2, $3 represent attributes of body symbols (RHS).

Production Rule	Semantic Rule / YACC Action
E -> E1 + T	E.val = E1.val + T.val ($$ = $1 + $3)
E -> T	E.val = T.val ($$ = $1)
T -> T1 * F	T.val = T1.val * F.val ($$ = $1 * $3)
T -> F	T.val = F.val ($$ = $1)
F -> digit	F.val = digit.lexval ($$ = $1)

(ii), (iii) & (iv) Bottom-Up Evaluation Trace & Attribute Propagation for: 2 * 3 + 4

The bottom-up parser (LR parser) evaluates synthesized attributes strictly during reductions from the leaves up to the root:

Step 1: Shift '2'
- Action: Reduce '2' via F -> digit
- Attribute synthesis: F.val = 2 ($$ = 2)
- Reduce F -> T: T.val = F.val = 2 ($$ = 2)

Step 2: Shift '*'

Step 3: Shift '3'
- Action: Reduce '3' via F -> digit
- Attribute synthesis: F.val = 3 ($$ = 3)

Step 4: Reduce T * F via T -> T1 * F
- Attribute synthesis: T.val = T1.val * F.val = 2 * 3 = 6 ($$ = 6)
- Reduce T -> E: E.val = T.val = 6 ($$ = 6)

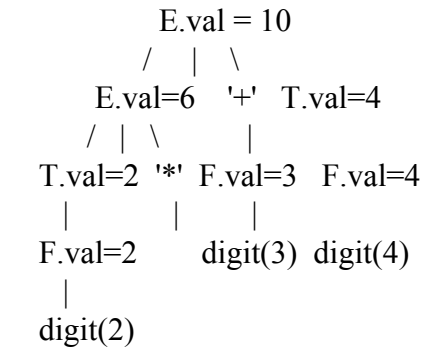
Step 5: Shift '+'

Step 6: Shift '4'
- Action: Reduce '4' via F -> digit

- Attribute synthesis: F.val = 4 ($$ = 4)
- Reduce F -> T: T.val = F.val = 4 ($$ = 4)

Step 7: Reduce E + T via E -> E1 + T
- Attribute synthesis: E.val = E1.val + T.val = 6 + 4 = 10 ($$ = 10)

Annotated Parse Tree (Synthesized Attribute Values at Each Node):



Final Evaluated Value synthesized at Root Node E.val = 10

—

QUESTION 4

Q4. A compiler developed using YACC must verify type compatibility between variables and user-defined types. The system distinguishes between name equivalence and structural equivalence during semantic analysis.
(i) Design a YACC grammar to process type declarations and variable declarations.
(ii) Implement semantic actions and symbol table routines to compare type expressions.
(iii) Differentiate and verify name equivalence and structural equivalence for the following declarations:
type A = int;
type B = int;
A x;
B y;
(iv) Test the parser with multiple type declarations and assignment statements.
(v) Display the type compatibility result and indicate whether the types are equivalent under:

(a). Name Equivalence
(b). Structural Equivalence (10 Marks)

ANSWER FOR QUESTION 4

(i) & (ii) YACC Grammar and Symbol Table Routines for Type Expressions

Symbol Table structure & Type checking routines:

```
struct SymbolTableEntry {
   char name[20];       /* Identifier name: A, B, x, y */
   char typeName[20];   /* Type string: "A", "B", "int" */
   char underlying[20]; /* Base representation: "int" */
} symTable[100];
int symCount = 0;

/* Name Equivalence Routine: Returns 1 if type names match exactly */
int checkNameEquivalence(char* type1, char* type2) {
   return strcmp(type1, type2) == 0;
}

/* Structural Equivalence Routine: Recursively expands aliases to fundamental primitives
*/
int checkStructuralEquivalence(char* type1, char* type2) {
   char* u1 = getUnderlyingPrimitive(type1);
   char* u2 = getUnderlyingPrimitive(type2);
   return strcmp(u1, u2) == 0;
}
```

(iii), (iv) & (v) Differentiation and Verification for Given Declarations

Declarations under analysis:

```
type A = int;
type B = int;
A x;
B y;
```

Detailed Semantic Analysis:

1. Analysis under Name Equivalence:
   - Definition: Two variables are type-equivalent if and only if they are declared with the exact same type name.
   - Variable 'x' has type name 'A'.
   - Variable 'y' has type name 'B'.
   - Comparison: Type name 'A' != Type name 'B'.

- Compatibility Result: NOT EQUIVALENT (Incompatible / Type Error during assignment x = y).

2. Analysis under Structural Equivalence:
- Definition: Two variables are type-equivalent if their fully expanded underlying type structures are identical.
- Type 'A' expands structurally to 'int'.
- Type 'B' expands structurally to 'int'.
- Comparison: Underlying structure 'int' == 'int'.
- Compatibility Result: EQUIVALENT (Compatible / Assignment x = y is permitted).

Summary Table of Compatibility Results:

Equivalence Rule	Type of x	Type of y	Compatibility Result
Name Equivalence	A	B	NOT EQUIVALENT (Incompatible)
Structural Equivalence	A (expands to int)	B (expands to int)	EQUIVALENT (Compatible)

QUESTION 5

Q5. A compiler implemented using YACC processes assignment statements involving multiple data types (int, float). During semantic analysis, the compiler performs automatic type conversion (coercion).
(i) Design a YACC grammar for variable declarations and assignment statements.
(ii) Analyze the following program fragment:
float temperature;
int sensor_value;
temperature = sensor_value;
(iii) Implement semantic actions and symbol table entries to support type promotion rules.
(iv) Demonstrate how the compiler performs implicit type conversion (int -> float) during assignment.
(v) Display the resulting type information and the final type of the variable after assignment. (10 Marks)

ANSWER FOR QUESTION 5

(i) & (iii) YACC Grammar and Type Promotion Semantic Actions

YACC specification for type declaration, assignment, and automatic coercion:

```
%{
#include <stdio.h>
#include <string.h>
enum DataType { TYPE_INT, TYPE_FLOAT, TYPE_UNKNOWN };

struct Symbol {
   char name[30];
   enum DataType type;
} symtab[50];
int sym_count = 0;

void addSymbol(char* name, enum DataType type) {
   strcpy(symtab[sym_count].name, name);
   symtab[sym_count].type = type;
   sym_count++;
}

enum DataType getSymbolType(char* name) {
   for(int i=0; i<sym_count; i++) {
      if(strcmp(symtab[i].name, name) == 0) return symtab[i].type;
   }
   return TYPE_UNKNOWN;
}
%}

%union { char id[30]; int type_val; }
%token <id> IDENTIFIER
%token FLOAT_KEYWORD INT_KEYWORD

%%
stmt_list:
   stmt_list stmt
  | stmt
  ;

stmt:
   FLOAT_KEYWORD IDENTIFIER ';' { addSymbol($2, TYPE_FLOAT);
printf("Declared %s as FLOAT\n", $2); }
```

```
  | INT_KEYWORD IDENTIFIER ';'   { addSymbol($2, TYPE_INT);   printf("Declared
%s as INT\n", $2); }
  | IDENTIFIER '=' IDENTIFIER ';' {
        enum DataType lhs_t = getSymbolType($1);
        enum DataType rhs_t = getSymbolType($3);
        printf("\nAnalyzing Assignment: %s = %s\n", $1, $3);
        if (lhs_t == TYPE_FLOAT && rhs_t == TYPE_INT) {
            printf("[Type Coercion Triggered]: Implicit conversion applied: int -> float\n");
            printf("[AST / Intermediate Code Generated]: %s = intToFloat(%s)\n", $1, $3);
            printf("Resulting Type of RHS: FLOAT\n");
            printf("Final Type of LHS Variable (%s): FLOAT\n", $1);
        } else if (lhs_t == rhs_t) {
            printf("Types match exactly. No conversion needed.\n");
        }
    }
  ;
%%
```

(ii), (iv) & (v) Analysis of Program Fragment and Coercion Execution

Input Code Fragment:

```
float temperature;
int sensor_value;
temperature = sensor_value;
```

Step-by-Step Compiler Processing:

1. Processing 'float temperature;':
   - Symbol Table entry added: Name = "temperature", Type = TYPE_FLOAT.

2. Processing 'int sensor_value;':
   - Symbol Table entry added: Name = "sensor_value", Type = TYPE_INT.

3. Processing Assignment 'temperature = sensor_value;':
   - LHS Variable Type lookup: temperature -> TYPE_FLOAT.
   - RHS Variable Type lookup: sensor_value -> TYPE_INT.
   - Semantic Check: Target type is FLOAT, source type is INT.
   - Promotion Rule: According to standard C compiler type conversion hierarchy, integers are implicitly widening-promoted to floating-point numbers without precision loss.
   - Code Generation Action: Insert implicit coercion operator node (intToFloat).

4. Final Output Displayed by Compiler:

   Declared temperature as FLOAT
   Declared sensor_value as INT

Analyzing Assignment: temperature = sensor_value
[Type Coercion Triggered]: Implicit conversion applied: int -> float
[AST / Intermediate Code Generated]: temperature = intToFloat(sensor_value)
Resulting Type of RHS Expression: FLOAT
Final Type of LHS Variable (temperature): FLOAT
